



QUALIDADE DE SOFTWARE MÉTRICAS COCOMO

Prof. Me. Júlio Vansan Gonçalves

MÉTRICA

- Em engenharia de software, **métrica significa medida quantitativa** usada para avaliar atributos de um projeto, processo ou produto.
- Sem métricas, tudo vira “achismo”.
- Exemplo: Imagine um gerente dizendo “acho que vamos terminar em 3 meses”
Isso é uma opinião.
- Já dizer “pelo histórico de projetos semelhantes, com 10.000 linhas de código, estimamos 6 meses”
Isso é baseado em métricas.

TIPOS DE MÉTRICAS EM SOFTWARE

- **Métricas de processo:** tempo gasto em cada fase, esforço da equipe, produtividade.
Ex.: quantas horas a equipe gastou em testes versus codificação.
- **Métricas de produto:** tamanho, complexidade, qualidade.
Ex.: número de defeitos por mil linhas de código (defects/KLOC).
- **Métricas de projeto:** cronograma, custo, esforço.
Ex.: horas de trabalho estimadas para entregar um módulo.

MÉTRICAS NO DIA A DIA

- **Pensem em uma métrica pessoal que usam no dia a dia**
- **Exemplos:**
- **Tempo de estudo por semana,**
- **Tempo média gasto no transporte,**
- **Número de páginas lidas por dia.**

- **Software também precisa ser medido.**

O QUE É COCOMO?

- **Definição:**
- **COCOMO = Constructive Cost Model**
Modelo de Estimativa de Custos para Software.
- **Criado por Barry Boehm nos anos 1980.**
- **Objetivo:**
Estimar esforço, tempo e custo de desenvolvimento de software.

POR QUE É IMPORTANTE?

- **Evita subestimar** projetos
- “achávamos que levaria 2 meses, mas levou 1 ano”.
- Ajuda a negociar **prazos realistas** com clientes.
- Dá **previsibilidade** para gestão de equipe e orçamento.

EXEMPLO PRÁTICO

- Uma empresa vai desenvolver um aplicativo de delivery.
- Antes de começar, precisa responder:
- Quantas pessoas vamos precisar?
- Quanto tempo vai levar?
- Quanto vai custar?
- É aí que entra o COCOMO: ele fornece um modelo matemático para **transformar estimativas vagas em números.**

TIPOS DE COCOMO

- COCOMO não é “uma fórmula única”, ele tem níveis de sofisticação:
- COCOMO Básico
- COCOMO Intermediário
- COCOMO Detalhado

COCOMO BÁSICO

- Usa apenas o tamanho do software em KLOC (milhares de linhas de código).
- Fórmula geral:
- Esforço (PM) = $a \times (\text{KLOC})^b$
- PM = Person-Months (meses-homens, esforço total)
- a e b = constantes definidas conforme o tipo de projeto.
- Depois o esforço é convertido em tempo (meses) e equipe (número de pessoas).

TIPOS DE PROJETOS NO BÁSICO

- **Orgânico:**
- Projetos simples, equipes pequenas e experientes, requisitos claros.
- Exemplo: Sistema interno de cadastro.
- Constantes:
 - $a = 2.4$
 - $b = 1.05$

TIPOS DE PROJETOS NO BÁSICO

- **Semidestacado:** Projetos de médio porte, com equipes mistas (alguns experientes, outros não), requisitos moderadamente complexos.
- Exemplo:
Sistema de controle para uma empresa de médio porte.
- Sistema de vendas online.
- Constantes:
 - $a = 3.0$
 - $b = 1.12$

TIPOS DE PROJETOS NO BÁSICO

- **Embutido** (embedded): Altíssima complexidade, restrições rígidas.
- Projetos grandes, fortemente restritos por hardware, regulamentos ou requisitos críticos.
- Exemplo: software de controle de voo de um avião.
- Constantes:
 - $a = 3.6$
 - $b = 1.20$

EXEMPLO PRÁTICO

- Se ele for Orgânico:
- $PM = 2.4 \times (32)^{1.05} \approx 91$ meses-homens
- Se ele for Embutido:
- $PM = 3.6 \times (32)^{1.20} \approx 177$ meses-homens
- Note como o mesmo tamanho (32 KLOC) pode exigir o dobro de esforço dependendo do tipo de projeto.
- Porque os valores de a e b refletem a complexidade real observada nos projetos que Boehm estudou.

COMPARANDO COCOMO BÁSICO

Tamanho (KLOC)	Orgânico	Semideterminado	Embutido
10.0	26.93	39.55	57.06
50.0	145.93	239.87	393.61
100.0	302.14	521.34	904.28

COCOMO INTERMEDIÁRIO

- Além do tamanho (KLOC), considera atributos do projeto, chamados de drivers de custo.
- Exemplos de drivers:
- Confiabilidade exigida do software (RELY).
- Restrição de tempo de execução (TIME).
- Experiência da equipe (ACAP).
- **Exemplo prático:** Se o software precisa rodar 24/7 sem cair (alta confiabilidade), **o esforço aumenta.** Se a equipe é experiente, **o esforço diminui.**

FÓRMULA NO COCOMO INTERMEDIÁRIO

- $PM = a \times (KLOC)^b \times EAF$
- PM = Person-Months (meses-homens)
- a e b = constantes (como no modelo básico, variam conforme o tipo de projeto)
- EAF (Effort Adjustment Factor) = fator de ajuste de esforço (multiplicador calculado a partir dos drivers de custo)

EXEMPLO DE ALGUNS DRIVERS (EXISTEM 15 NO COCOMO INTERMEDIÁRIO):

- **RELY** (Required Software Reliability)
- Confiabilidade exigida do software
- Baixa (nominal = 1.0): se cair de vez em quando não tem problema (ex.: app de entretenimento).
- Muito alta (ex.: 1.4): software precisa rodar 24/7 sem falhar (ex.: controle de tráfego aéreo).

EXEMPLO DE ALGUNS DRIVERS (EXISTEM 15 NO COCOMO INTERMEDIÁRIO):

- **TIME** (Execution Time Constraint)
- Restrição de tempo de execução
- Nominal: não há grandes restrições.
- Alta (ex.: 1.3): precisa rodar em tempo real, com resposta rápida.

EXEMPLO DE ALGUNS DRIVERS (EXISTEM 15 NO COCOMO INTERMEDIÁRIO):

- **ACAP** (Analyst Capability)
- Capacidade/experiência da equipe de analistas
- Muito baixa (ex.: 1.4): equipe inexperiente, pouca vivência.
- Muito alta (ex.: 0.7): equipe sênior, reduz esforço porque trabalha com mais produtividade.

EXEMPLO DE ALGUNS DRIVERS (EXISTEM 15 NO COCOMO INTERMEDIÁRIO):

- **STOR** (Storage Constraint)
 - Restrição de armazenamento
 - Quanto mais limitado o ambiente, mais esforço é exigido.
- **TOOL** (Use of Software Tools)
 - Uso de ferramentas de suporte ao desenvolvimento
 - Se a equipe usa ferramentas modernas e boas, o esforço cai.

EXEMPLO PRÁTICO:

- Um sistema de 30 KLOC.
- Projeto Orgânico
constantes: $a=3.2$ e $b=1.05$.
- Sem drivers de custo
 $PM=3.2 \times (30)^{1.05} \approx 108PM$

EXEMPLO PRÁTICO:

- Agora, aplicando drivers:
- Alta confiabilidade exigida ($RELY = 1.15$)
- Restrições de tempo de execução ($TIME = 1.10$)
- Equipe muito experiente ($ACAP = 0.85$)
- $EAF = 1.15 \times 1.10 \times 0.85 \approx 1.08$
- $PM_{ajustado} = 108 \times 1.08 \approx 117PM$

COMPARANDO COCOMO INTERMEDIÁRIO

Driver	Muito Baixo	Baixo	Nominal	Alto	Muito Alto	Extra Alto
RELY (Confiabilidade)	-	0.75	1.00	1.15	1.40	-
TIME (Restrição de tempo)	-	-	1.00	1.11	1.30	-
ACAP (Capacidade dos analistas)	1.46	1.19	1.00	0.86	0.71	-
STOR (Restrição de armazenamento)	-	-	1.00	1.06	1.21	1.56
TOOL (Uso de ferramentas)	1.24	1.10	1.00	0.91	0.82	-

COCOMO DETALHADO

- Estima o esforço total (como no intermediário, com fórmula e drivers).
- Divide esse esforço entre fases do projeto, usando percentuais definidos por Boehm.
- As fases típicas são:
 - **Análise e planejamento** (requirements, especificação)
 - **Projeto** (design)
 - **Codificação** (coding)
 - **Testes e integração**

EXEMPLO DE DISTRIBUIÇÃO (TÍPICA)

- **Para um projeto orgânico (mais simples, equipe experiente), Boehm sugeriu valores aproximados:**

Fase	Percentual do esforço total
Análise	6% – 8%
Projeto (design)	16% – 18%
Codificação	50% – 55%
Testes/Integração	25% – 26%

- **Já em um projeto embutido (embedded), que é mais crítico e cheio de restrições, a análise e os testes pesam muito mais (pode chegar a 40% só em testes!)**

ONDE SE APLICA O COCOMO DETALHADO

- **Sistemas militares:**
Onde falhas podem custar vidas, então testes e validações são enormes.
- **Sistemas bancários:**
Segurança, disponibilidade 24/7 e auditoria exigem altíssimo esforço em testes e análise de requisitos.
- **Aviação, medicina, automotivo:**
Software embarcado crítico, com certificações rígidas.

LIMITAÇÕES DO COCOMO:

- Baseado em linhas de código (KLOC)
Difícil prever no início.
- Não considera metodologias ágeis (Scrum, Kanban) diretamente.
- Precisa de histórico de projetos para calibrar.

BOAS PRÁTICAS:

- Usar COCOMO como ponto de partida, não como verdade absoluta.
- Atualizar a estimativa conforme o projeto avança.
- Comparar sempre com dados reais da equipe.

DIFERENÇA ENTRE: BÁSICO, INTERMEDIÁRIO E DETALHADO

- **Básico:**
Só considera o tamanho do software (KLOC).
- **Intermediário**
Adiciona drivers de custo (confiabilidade, experiência, restrições, etc.).
- **Detalhado**
Além disso, distribui o esforço entre fases específicas do desenvolvimento.

